Generating learnable Terminal-Bench 2 tasks

A pipeline that designs, builds, checks and measures tasks for glm-5.3-flash, and what two days of measurement taught us

Generator: glm-5.1. Evaluator: terminus-2 + glm-5.3-flash, reasoning effort high, 5 runs per task. Learnable = 1 to 3 of 5 runs pass, every failure the model's fault.

Based on CHANGELOG.md (the full decision record) as of 22 October 2026, 17:00. Several measurements were still running when this was written; they are marked as such.

The job and the bar it is measured against

Produce tasks that are hard but solvable and fair, in the standard Harbor format, at a difficulty where glm-5.3-flash passes 1 to 3 times in 5. Four deliverables (README parts):

- Part 1: a generator that turns a topic into a valid, correct task (structure, oracle solution scores 1, no-op scores 0).
- Part 2: a band evaluation that measures each task 5 times and keeps only learnable ones, with every failure explained and the model's fault.
- Part 3: a batch of 10+ topics with a funnel (structural pass, correct, learnable), yield, cost and time.
- Part 4: robustness: an audit record of every step, resume after a kill without re-paying, two copies at once.

Constraints that shaped every decision: tasks must be novel; a failing run must be the model's fault (timeouts do not count); tests must not be gameable and answers must not leak into the environment; cost per task must stay reasonable.

The hard part turned out to be none of the plumbing. It was inducing difficulty for this model without making a task unfair.

The pipeline as it stands

Stage	What happens	Gate / record
Seed card -> design	A YAML card per topic (shape, what the agent cannot verify, dial, quirk matrix, evidence) binds the design. glm-5.1 writes a spec with 8-20 independent hidden checks.	Design gate, seed gate, reference-access and answer-source gates (code). One critic call red-teams and rewrites.
Files	Project files first (manifest, continuation calls if a reply is cut), then instruction, Dockerfile, solution, tests against the real code.	Instruction review (leaks, study guide, contract); files gate (oracles, reference copies, inferred rules in documents).
Correctness	Structural validator; harbor oracle run must score 1; no-op run must score 0; a pipeline-written tamper guard test.	Repair loop with the real error fed back (bounded).
Mutants	Each hidden check's plausible wrong fix is run as the solution; its named test must fail it.	Survivors recorded as test weaknesses (tolerance 1) or repaired.
Band eval	5 runs of terminus-2 + glm-5.3-flash; timeouts and infrastructure errors are invalid and re-run; early stops only where the verdict is certain.	Per-test table, trajectory scan for tampering (a hacked pass scores 0).
Review and verdict	One glm-5.1 call judges each failure (model or task fault, with the sentence the run broke) and writes the lesson.	VERDICT.md, band.json, batch report with funnel, yield, cost, time; confirmation eval for a learnable task.

Robustness (Part 4): a batch ledger with per-topic locks and an audit log; every model reply cached so a resumed run replays finished calls free; orphaned harbor jobs adopted rather than killed; 65 offline checks (kill mid-generation, kill mid-eval, two copies, races). A live dashboard shows stages, calls, agent trajectories and verdicts.

What was measured: about forty 5-run evaluations

Task shape	Examples	Result
Bug hunt (planted faults in a project)	SQLite migration, nginx routes, CMake ABI, Pillow upgrade, 14 variants with hints, references and oracles on or off	5/5 at every setting; 4-9 steps at first, 16-45 steps once faults were interactions, same verdict
Hand-built hard reference	Static-site builder with five interaction faults and no confessing comments	4-5/5 at 42-44 steps: every findable fault was found
Specification (12-20 stated rules)	Markdown table formatter, packing slip, statement renderer	4-5/5: a precise, testable spec is a checklist to this agent
Judgement (property violated at several sites)	Billing rounding, HTTP header injection, payment webhook	In band only through task faults; 3/3 once fixed
Long road / domain knowledge	Six chained build obstacles; EBCDIC mainframe extract	3/3 and 5/5: it fixes by reading; it knows the formats
Executable reference (binary it can run, not read)	awk report port, byte-identical on 200 hidden files	5/5 and 4/4: it fuzzes its port against the reference until they match
Rule induction from data	Address normaliser (4 settings), IDS rule recovery	Address 4-5/5 at every fair setting; IDS with a 2-hour budget: 2 passes, 1 fair failure, 2 running (169-228 steps)
Model fitted to held-out data	Island microgrid demand forecast	1/5; all four failures on one test that passes the truth by 4% against another season

Pipeline evaluations logged: 16 (14 too easy, 2 in band through task faults later fixed); hand-built and hand-edited evaluations: about 30 more. The IDS rule-recovery task is the first on which fair runs disagree for reasons of difficulty.

The one pattern behind every result

Whatever the agent can read, already knows, or can run to compare against, it checks and gets right.

- It reads every file before acting, forms hypotheses, writes its own end-to-end tests for every case it can think of, and iterates until they pass. Hidden faults are found by reading; a numbered spec is tested item by item.
- More of anything (faults, lines, layers, chained obstacles) changed the step count, never the verdict.
- A reference it can run is an answer key: on the awk port every run compared its output with awk until nothing differed, including the exact edge cases a harder hidden test was meant to catch.
- A rule written in a document is an answer key: the address task's README listed every rewrite rule and their order; the microgrid notes gave the heating slope, the ferry bump and the holiday factors with worked examples.
- When the graded data comes from the same process as the practice data, matching the practice set fully determines the score: the address runs mined the rules by diffing all 900 pairs and iterated to 900/900, three times, at every setting.
- The only fair failures came where careful work still leaves something to judge: an obvious fix that is incomplete, a boundary the data straddles but never lands on, a holiday effect shown once.

Each day's rules closed one route at a time. The record of what was tried and discarded is as important as what stayed.

Decisions, part 1: building and measuring (21 October)

Decision	Alternative considered	Why, and what happened next
Two-stage generation: design spec, then files; Python owns task.toml, test.sh and the canary	One call writing everything	The model commits to the hidden state and tests before code; boilerplate mistakes could only cause validator failures. Kept.
Repair loop fed with the real error (bounded rounds)	Fail fast and regenerate	Most first drafts fail on small things; a repair is far cheaper than a fresh generation. Kept, later bounded harder.
Band eval with timeouts as invalid and re-run; glm-5.1 triage of every failure	Count timeouts as failures; rule-based reasons	README: timeouts are not the model failing; only a reader of the trajectory can tell misdiagnosis from ambiguity. Kept; triage must quote the sentence the run broke.
Gates in code plus a critic, after an A/B showed prompt wording changed designs only on paper	Prompt wording alone	Free, objective checks before any Docker or model spend; the critic adds the agent's-eye view. Kept; the gates were rewritten twice.
Fault quotas (faults, layers, decoys, size floors) and a self-tuning difficulty dial	Pick levers by intuition	Measured: quotas produced filler, not difficulty; the dial saturated. Both removed on 21/22 October.
Hand-build one hard task before changing prompts again	Keep iterating prompts	Showed that hiding faults doubles the work but a careful agent still finds every findable fault. The lesson set the next day's direction.
Ledger, cached replies, adopted jobs, per-topic locks (Part 4)	Re-run from scratch after a kill	A resumed run pays only for the call in flight at the kill; two copies share a batch safely. Tested with 65 offline checks.

Decisions, part 2: where difficulty comes from (22 October)

Decision	Alternative considered	Why, and what happened next
Fairness = derivable: a test must follow from the stated goal, domain knowledge and the environment; the instruction must not list the traps	Spell every check out	A listed check is a checklist the agent ticks; an unstated one is unfair. Triage, review and critic all use this rule.
No reference in the environment; mutation check; confirmation eval	Trust the design	Agents copied the correct replay logic and fuzzed against verify commands; a lucky 2/5 is not learnable until it repeats.
Genres (spec, judgement, long road, domain, executable reference), then one recipe for every topic	Keep tuning one shape	Every shape failed the same way; the design space was the problem. The recipe: an evaluation the agent cannot reproduce in full, ground truth computed by code the tests run.
Seed cards: a binding card per topic (shape, what the agent cannot verify, dial, quirk matrix, evidence)	One-line topics	The generator kept choosing checkable designs; the card fixes the decisions that set difficulty before any spend. Gate rejections fell from about one per run to none.
Reference access: the agent never runs code whose output answers the graded question; frozen practice pairs instead	Keep the runnable reference, hide the source	Hiding the source still allows unlimited comparisons (awk 4/4). Retired the executable-reference recipe.
Inferred tasks: rules never stated in documents; evidence for every hidden case in the data (enforced by a test); a stated self-check gap; no study guide in the instruction	Rely on the critic	The critic passed a README that listed every rule; the generated instructions carried the quirk matrix as advice. Each rule is now a gate or a pipeline-written test, not a hope.
Reward gaming scored, not assumed: guard test, trajectory scan, hacked pass = failure counted apart	1/0 scoring	Harbor grades in the agent's container; an import hook would survive into grading. None of today's passes showed any sign.

Decision	Alternative considered	Why, and what happened next
Need-to-know writing: instruction and documents written by a call that never sees the hidden checks; the card copied into the design verbatim	Skip the spec and critic, write files straight from the card	The leaks were written in the files stage by a writer that knew the checks (the key-rotation instruction spelled out the fix twice). The critic restored the card's intent on 7 of 10 designs, so it stays; the single 60-80k-token reply is the failure mode to shrink, not grow.

A worked example: the address normaliser, three fair measurements

Version	Result	What the runs did
Rules and their order stated in the README (as generated)	5/6 passed, 9-21 steps	Typed up the README; the one failure missed a rule the README stated (model's fault).
README cut to the file format; one stated fact added for a case the data cannot show	4/5	Read all 900 pairs, mined the rules by diffing, iterated to 900/900. The failure expanded ST and DR inside names although 30 pairs show them kept (model's fault).
Rare rules at 1-3 training examples (was 6-68), held-out set weighted to them, dial 95%	4/5, then 5/5 with the Tips section removed	A rule seen once is found as surely as one seen fifty times; the rare rules were unambiguous from one example.

Why it matters: the held-out pairs come from the same generator as the training pairs, so reproducing the training set determines the hidden score. Rule induction over a 21-rule regex engine is below this model's level at any fair setting. The lesson became a design rule (self-check gap) and a seed-card field, and the topic was dropped from iteration.

Contrast: the IDS task, built the same way, kept four runs busy for 100-134 steps on the inclusive 60-second window rule and timed out at 60 minutes; its 2-hour-budget rerun is in progress. Reproducing 40,000 labels does not pin down a threshold's exact boundary, and that is where the difficulty lives.

What fairness required, learned the hard way

- **Every hidden test case needs evidence, not just every rule.** Removing stated rules exposed a whitespace-only line (address) and a UDP flow over the exfiltration threshold (IDS) that nothing visible supported; two otherwise excellent IDS runs failed only that case after reproducing all 40,000 labels. Now: each hidden case kind appears in the practice data at least 3 times, and a pipeline-required test fails the oracle if it does not.
- **Facts the data cannot show are stated once, qualitatively.** The microgrid heating response below the training temperatures; nothing else from the simulator.
- **Relative tests must compare within the same regime.** The microgrid long-weekend test compares a cold December Friday with warm autumn Fridays and passes the truth by 4%; three independent runs with 3% cold-snap error all failed it, the one pass took a separate long-weekend factor. The rule now: same-period baselines and a 10% margin for the oracle.
- **Every failing run failing the same assertion is the tell.** It flagged the three unfair Markdown tests, the webhook's unasked-for output, and this one.
- **A study guide is a leak.** 'Rule order matters', 'pay attention to rare pairs', 'the public tests do not cover boundaries' are the quirk matrix rewritten as advice. The instruction now states the job, interface, acceptance criterion, stated facts and the tamper sentence, nothing else.

Cost, time and operations

Item	Measured
Generation (glm-5.1)	About 20 minutes of design per task (median of 6 completed designs: 21 min), almost all of it two model calls of 5-25 minutes; calls that never answer are the largest risk (no wall-clock limit per call yet). 344 calls on 22 October, $21.46 reported; about $8 of batch reports and $3 of hand experiments the day before.
Evaluation (glm-5.3-flash)	Cheap: about $0.002-0.05 per run; a 5-run eval of a too-easy task costs cents. Long tasks (IDS) cost more in time than money: 60-120 minutes per run.
Build and checks	Validator seconds; oracle and nop 2-5 minutes; mutants 5-12 minutes typical, up to 45 with repairs. Repair rounds were the biggest time sink of the afternoon (3-4 rounds on IDS and microgrid) until mutant tolerance.
Gateway	Reasoning-effort settings ignored for glm-5.1; failures rose to 24% of calls for 90 minutes one morning and recovered; no throughput limit found up to 7 concurrent calls (per-call speed about 15% lower at 3-4 concurrent, time of day mattered far more).
Resume and audit	Killing a batch mid-eval and resuming re-paid nothing but the call in flight; every verdict has a VERDICT.md with stages, per-test results, triage and the lesson.

Where things stand (22 October, 17:00)

- **The first fair in-band candidate.** IDS rule recovery with a 2-hour budget, no study guide and complete evidence: 2 passes (169 and 215 steps), 1 failure that put the rate threshold at 50-or-more instead of more-than-50 (the data straddles it: the model's fault), 2 runs still going. Both passes were earned by inference, verified against the trajectories, the image and the solution.
- **Everything else fair was too easy or undecided by the task:** address 5/5 at its last setting; microgrid 1/5 with every failure on one thin test; the earlier shapes 4-5/5.
- **The ten-topic batch under all of today's rules** (started 15:10-15:30): the critic rewrote 7 of 10 designs citing the day's rules; two instructions rewritten by the review; one generation failure (the Go checkpoint race, three collapsed spec replies); excise the first to clear every check and enter its eval at 16:57; the rest in files, repairs or mutants. Full funnel expected 19:30-20:15.
- **Design calls are slower and fail more than this morning** (spec 9-30 min, critic 6-43 min; 10 of 48 replies unusable): every rule became a field the model writes and the critic re-emits, and long replies fall into the gateway's slow lane. Not throttling: per-call speed did not fall with concurrency.
- **Running now:** the IDS 2-hour eval, the batch, the Tcl/Tk port (first UI-parity build), and a microgrid build under need-to-know writing, the first test of the newest change.

Limitations, stated plainly

Limitation	Effect	Mitigation in place
Difficulty is still not controllable on demand	One fair in-band candidate (IDS) after two days; the 10-topic funnel will report what the latest rules achieve	Seed cards with a dial and a stated self-check gap; need-to-know writing; stretch goal A exists but is off by default and was tested once
Same-container grading	An agent could in principle plant an import hook that survives into the verifier	Guard test, trajectory scan, hacked pass scored 0; fresh-container grading would need a Harbor change
Agent time budget is a scaffold constant (60 min)	Long legitimate tasks time out and the eval is inconclusive (IDS)	Hand-set to 2 hours for IDS; deriving it from the design is not implemented
Long model replies are the biggest time sink	Replies of 40-100k tokens take 20-45 minutes, fall into the gateway's slow lane, hit the output cap with all tokens spent on hidden reasoning, or come back as broken JSON (10 of 48 in the batch window)	Reasoning budget per stage (advisory), lenient JSON repair, compile check before Docker, automatic retries; a per-call wall-clock limit and a critic that returns a patch instead of a full spec are not implemented
LLM checks miss what gates catch	The critic passed a README that listed every rule; the review missed a study guide; the files writer leaked twice on one design	Each lesson became a code gate or a pipeline-written test; documents are now written by a call that cannot know the hidden checks
Held-out predictability	When graded data comes from the practice process, matching the practice set fixes the score	self_check_gap required; topics chosen where boundaries, windows and response curves are not pinned by training
Five runs per verdict	A 2/5 can be luck; verdicts are noisy near the band edges	Confirmation eval for any learnable task; early stops only where the verdict is certain
Cost of exploration	About $35 of model spend over two days, most of it generation; many hand-built experiments	Cheapest gates first; cached replies; bounded retries with switches

Next steps, in order of expected value

- **Read the ten-topic funnel.** It is the first batch under every rule learned today; its report (structural, correct, learnable, yield, cost, time) says whether the rules moved the yield. Any learnable task gets its confirmation eval.
- **Agent budget from the design.** IDS needed two hours where the scaffold gives one. The critic estimates the steps; set the task's agent timeout to twice that (floor 1 hour, ceiling 2), and rerun a timed-out eval with a doubled budget instead of the same one.
- **Shorter model replies:** a wall-clock limit per call (abort and retry at 30 minutes), and a critic that returns a patch of the fields it changes instead of re-emitting the whole design; the two together address most of the batch's lost time.
- **Measure need-to-know writing** on the microgrid build now running, then on the next batch: do the generated documents stay clean without the review having to rewrite them?
- **Fix the microgrid long-weekend test** to a same-week comparison and re-measure; apply the margin rule to every relative test the generator writes (a post-oracle check that the oracle clears each bar by 10%).
- **More topics of the IDS kind:** numeric thresholds, inclusive windows, time-zone rules, response curves, where reproducing the practice data does not pin down the hidden result. Fewer of the address kind.
- **Use stretch goal A on tasks that land 4/5:** one fair, recorded edit (a tighter dial, a thinner margin the oracle still clears) and a re-measurement, which is cheaper than a new design.
- **Measure the critic against outcomes:** a report column comparing its flags with the band result, since its verdicts have never been scored.
- **Fresh-container grading** if Harbor allows it, which removes the same-container exposure instead of detecting it.